

Day08-LVM逻辑卷与NFS网络存储

课程目标

- 理解 LVM 基本概念和架构
- 掌握 LVM 的创建和管理
- 学会扩展和缩减逻辑卷
- 理解存储堆栈层结构
- 掌握 NFS 网络存储配置
- 学会使用 autofs 自动挂载

上午 - LVM逻辑卷管理

课前故事：LVM 的诞生

为什么需要 LVM(Logical Volume)?

在 LVM 出现之前，系统管理员面临着一个严峻的挑战：

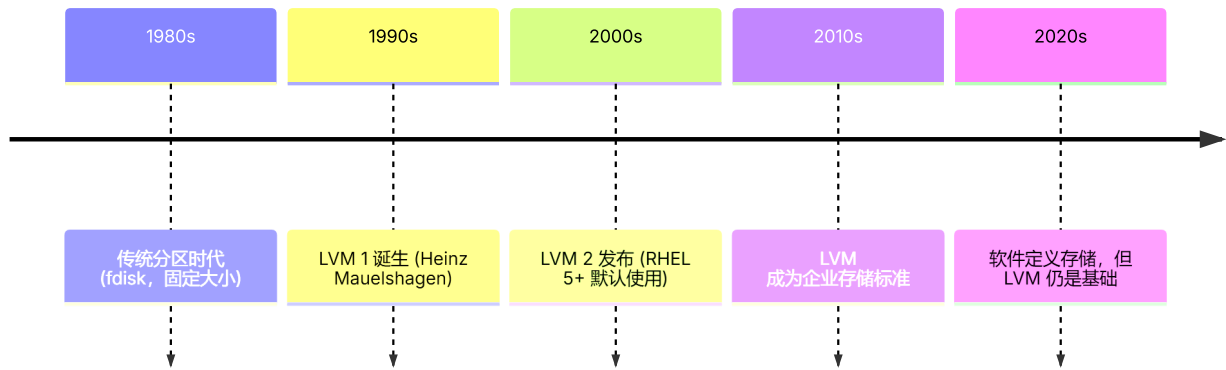
服务器运行时，某个分区空间不足，但相邻分区还有大量空闲空间...

传统分区的"硬墙壁"：

- 想扩展分区？必须先备份数据，删除分区，重新创建，再恢复数据
- 想跨磁盘使用空间？不可能，每个分区必须完整位于一个磁盘上
- 想在线调整大小？抱歉，需要停机维护

存储技术的演进历史：

存储管理技术发展史



LVM 的革命性设计:

LVM 引入了"存储池"概念, 彻底改变了磁盘管理:

传统分区模式:

```
磁盘 └─ 分区1 (10GB) ─ 固定大小 ─ 无法扩展
      └─ 分区2 (20GB) ─ 固定大小 ─ 无法扩展
      └─ 分区3 (30GB) ─ 固定大小 ─ 无法扩展
```

LVM 模式:

```
磁盘1 └─
磁盘2 └─ 存储池 (VG) ─ 灵活分配 ─ 在线扩展
磁盘3 └─
      ↓
    可动态调整的逻辑卷
```

LVM 带来的核心价值:

1. 灵活性 - 在线扩容/缩容(xfs格式不能缩容)
2. 聚合性 - 多块磁盘统一管理
3. 效率性 - 空间利用率最大化
4. 可靠性 - 支持快照和镜像

第一部分: LVM基础概念

为什么需要学习 LVM?

通过课前故事，我们了解了 LVM 如何解决传统分区的痛点。现在让我们深入学习 LVM 的实际使用。

本部分将学习：

- LVM 的三层架构 (PV/VG/LV)
- PE/LE 的概念和作用
- LVM 的创建和管理流程
- 逻辑卷的扩展和缩减操作

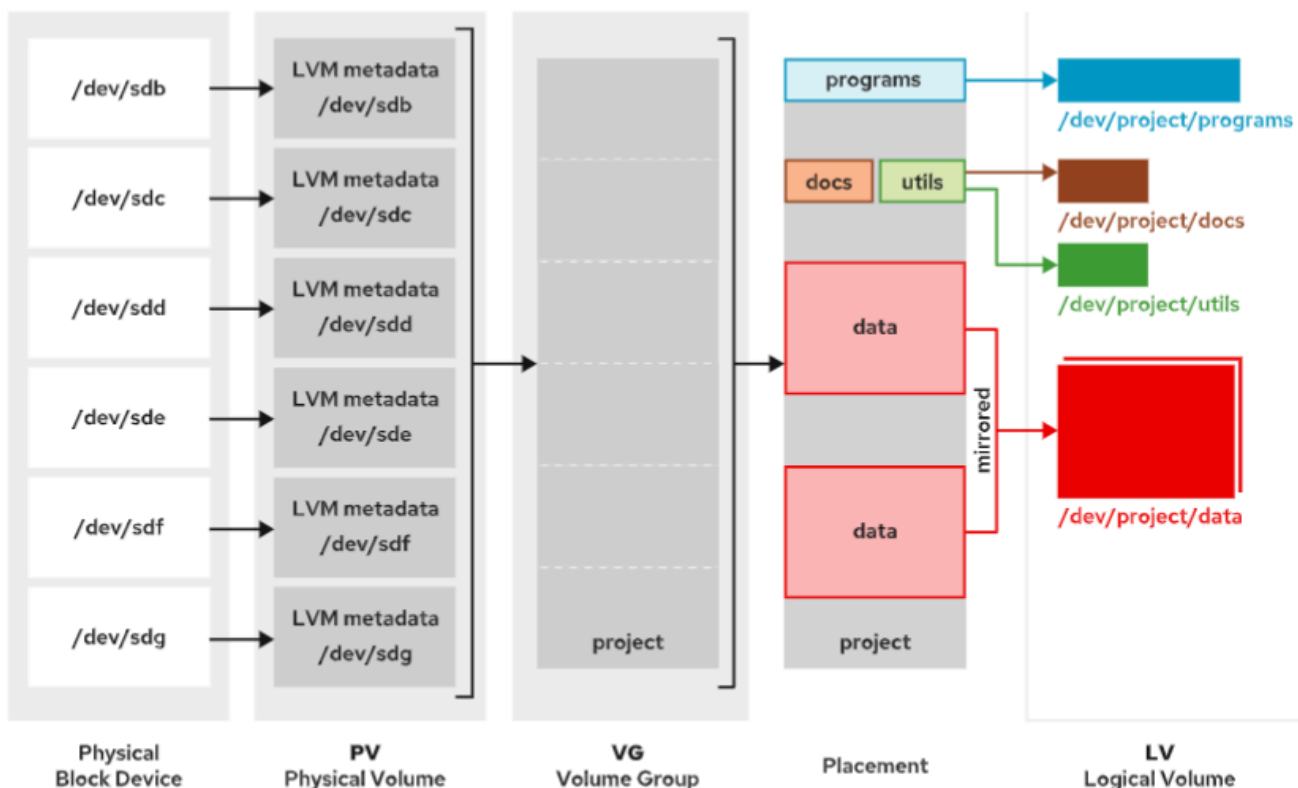
掌握这些知识，你将能够：

- 灵活管理服务器存储空间
- 在线调整文件系统大小
- 创建存储快照进行数据备份

1.1 什么是 LVM

LVM (Logical Volume Manager) :

逻辑卷管理器，提供灵活的磁盘存储管理



- 将多个物理磁盘组合成存储池

- 动态调整逻辑卷大小
- 在线扩展文件系统
- 支持快照和镜像

传统分区 vs LVM:

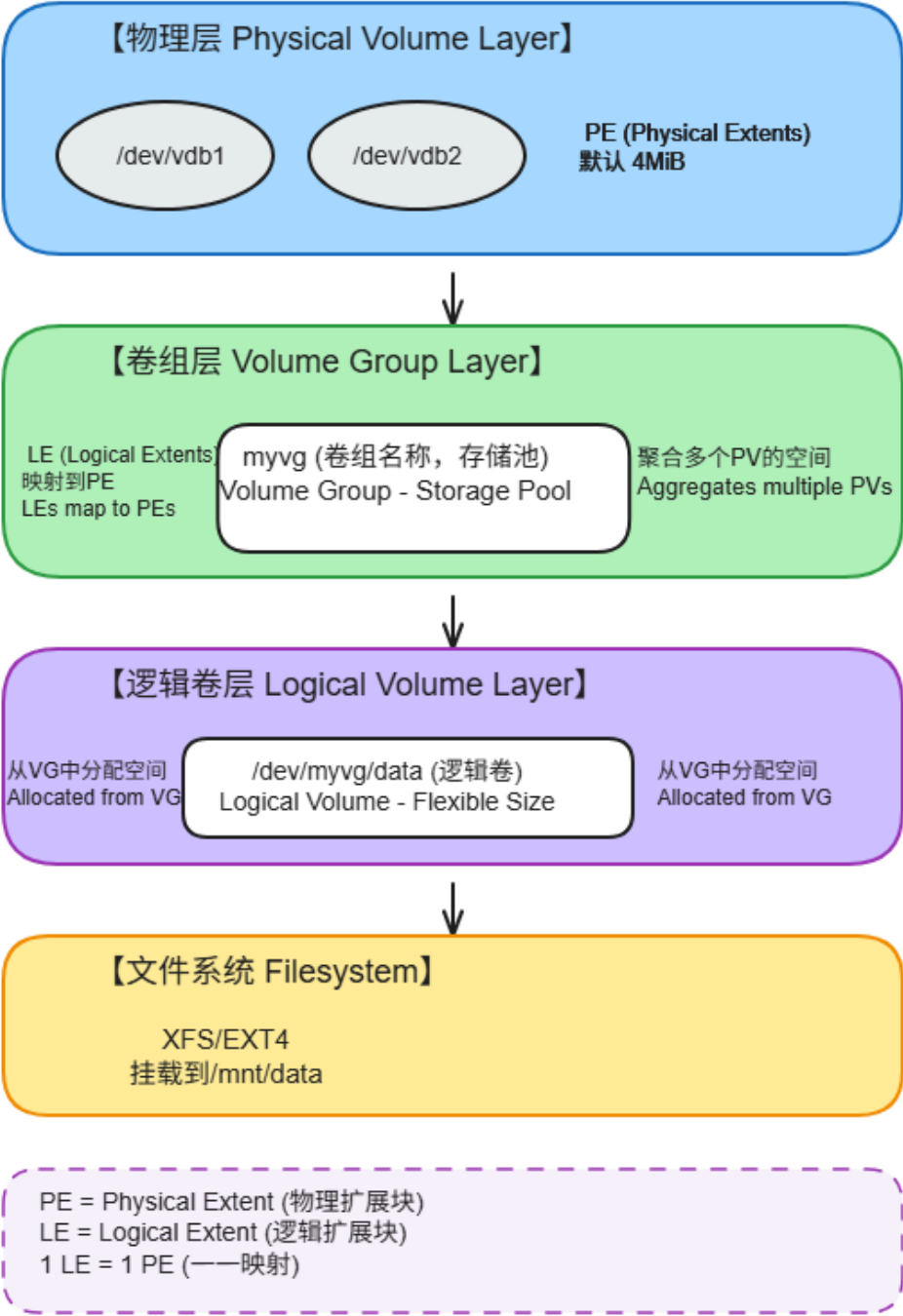
特性	传统分区	LVM
空间调整	困难	容易
跨磁盘	不支持	支持
大小限制	固定	动态

1.2 LVM 架构

LVM架构图示:

LVM 三层架构

LVM Three-Tier Architecture



LVM 三个层次:

物理卷(PV) → 卷组(VG) → 逻辑卷(LV)

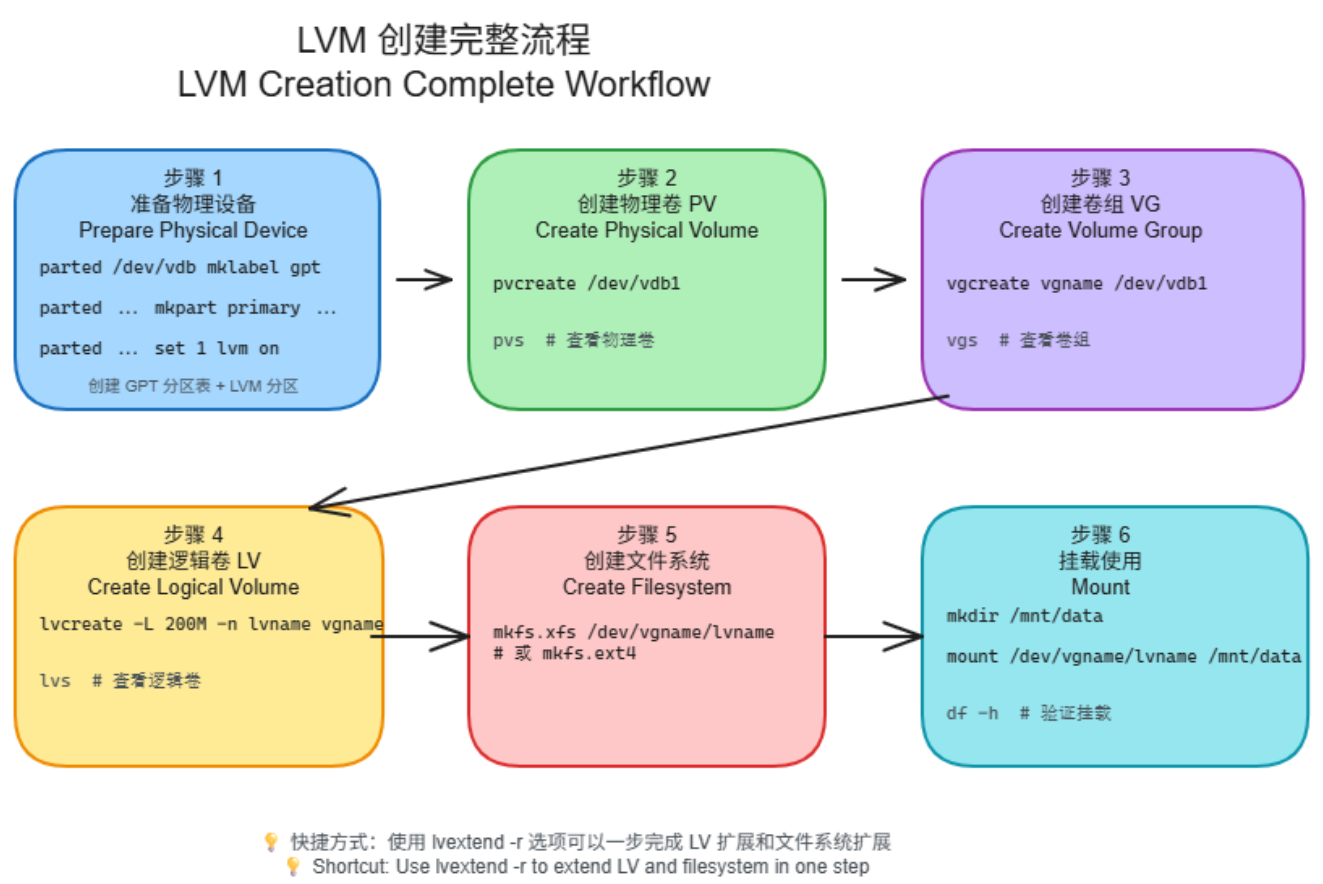
层次	说明	示例
PV (Physical Volume)	物理卷, 初始化的块设备	<code>/dev/vdb1</code>
VG (Volume Group)	卷组, PV的集合	<code>vg01</code>
LV (Logical Volume)	逻辑卷, 从VG分配	<code>/dev/vg01/lv01</code>

PE (Physical Extent) 和 LE (Logical Extent):

- PE：物理区块，LVM最小存储单位（默认4MiB）
- LE：逻辑区块，映射到PE
- 1个LE = 1个PE

1.3 LVM 工作流程

LVM创建流程图：



创建 LVM 的步骤：

块设备 → 创建PV → 创建VG → 创建LV → 格式化 → 挂载

SHELL

第二部分：创建LVM逻辑卷

流程图：

```
/dev/vdb1, /dev/vdb2
  ↓ pvcreate
vg01 (卷组)
  ↓ lvcreate
/dev/vg01/data (逻辑卷)
  ↓ mkfs
文件系统
  ↓ mount
/mnt/data (挂载点)
```

2.1 准备物理设备

使用 **parted** 创建分区：

```
# ***** 分区创建 *****

# 步骤1: 启动 parted 分区工具
# parted 是功能强大的分区工具, 支持 GPT 和 MBR 分区表
# /dev/vdb 是要分区的目标磁盘设备
parted /dev/vdb

# 交互式命令在 parted 提示符下执行:

# 步骤2: 创建 GPT 分区表
# GPT (GUID Partition Table) 是现代标准分区表类型
# 优势: 支持 >2TB 磁盘、支持最多 128 个主分区、数据冗余备份
# mklabel = 创建/修改分区表命令
# gpt = GPT 分区表类型 (另一个选项是 msdos, 即传统的 MBR)
mklabel gpt

# 步骤3: 创建主分区
# mkpart = 创建分区的命令
# primary = 分区类型 (主分区)
# 1MiB = 分区起始位置 (从磁盘 1MiB 处开始, 留出对齐空间)
# 512MiB = 分区结束位置 (创建一个约 511MiB 的分区)
mkpart primary 1MiB 512MiB

# 步骤4: 设置分区1为 LVM 类型
# set = 设置分区标志
# 1 = 分区编号 (刚才创建的第一个分区)
# lvm on = 启用 LVM 标志 (系统会将该分区识别为可用于 LVM 的物理卷)
set 1 lvm on

# 步骤5: 设置显示单位为 MiB (便于查看)
# unit = 设置显示单位
# MiB = 使用 Mebibytes (1024×1024 bytes) 作为单位
unit MiB

# 步骤6: 打印分区表查看结果
# print = 显示当前分区表信息
print

# 步骤7: 退出 parted
# quit = 退出 parted 工具, 保存所有更改
quit
```

参数说明:

- `mklabel gpt` = 创建 GPT 分区表 (现代磁盘推荐)
- `mkpart` = 创建分区命令
- `primary` = 主分区类型 (GPT 不区分主/扩展/逻辑)
- `1MiB` | `512MiB` = 分区起止位置 (使用 MiB 单位确保对齐)
- `set 1 lvm on` = 设置分区1的 LVM 标志位 (hex code: 8e)

为什么使用 **MiB** 而不是 **MB**?

MiB (Mebibyte) = 1024×1024 bytes = 1,048,576 bytes

MB (Megabyte) = 1000×1000 bytes = 1,000,000 bytes

使用 MiB 可以确保分区正确对齐到 4K 扇区边界, 提高性能

parted 非交互式写法 (脚本使用) :

SHELL

```
# ***** 分区创建 (脚本模式) *****

# 使用脚本模式 (非交互式) 创建分区
# 每个命令用单引号括起, 用空格分隔
# 适合在 shell 脚本中使用
parted /dev/vdb mklabel gpt \
parted /dev/vdb mkpart primary 1MiB 512MiB \
parted /dev/vdb set 1 lvm on

# 或使用 -s 选项避免交互确认
parted -s /dev/vdb mklabel gpt
parted -s /dev/vdb mkpart primary 1MiB 512MiB
parted -s /dev/vdb set 1 lvm on
```

2.2 创建物理卷

pvccreate - 初始化物理卷:

```
# ***** 物理卷管理 *****

# 步骤1: 创建物理卷 (Physical Volume)
# pvcreate = 将块设备初始化为 LVM 物理卷
# /dev/vdb1 = 目标分区 (必须是 lvm 类型的分区)
# 该命令会在分区末尾写入 LVM 元数据标签
pvcreate /dev/vdb1

# 输出说明:
# Physical volume "/dev/vdb1" successfully created
# 表示物理卷创建成功

# 步骤2: 查看物理卷详细信息
# pvdisplay = 显示物理卷的完整详细信息
# 包括: PV名称、VG名称、大小、PE数量、UUID等
pvdisplay

# 步骤3: 简短查看物理卷
# pvs = 以表格形式简短显示物理卷信息
# 适合快速查看多个 PV 的状态
pvs

# 输出示例:
# PV          VG      Fmt  Attr PSize  PFree
# /dev/vdb1    lvm2  a--  511.00m 511.00m
# ↑           ↑      ↑    ↑    ↑      ↑
# 设备        所属VG 格式 属性 总大小 空闲空间
#
# 字段说明:
# PV      = 物理卷设备路径
# VG      = 所属卷组 (空表示未加入任何卷组)
# Fmt     = LVM 格式版本 (lvm2 是当前标准)
# Attr    = 属性: a=可分配, w=可写, -=无此属性
# PSize   = 物理卷总大小
# PFree   = 可用空间 (未分配给LV的空间)

# 步骤4: 扫描系统中的物理卷
# pvscan = 扫描并显示系统中所有物理卷
# 在添加新磁盘后可用于刷新 LVM 缓存
pvscan
```

pvdisplay 输出字段详解:

字段	说明
PV Name	物理卷设备路径
VG Name	所属卷组名称
PV Size	物理卷总大小
PE Size	物理区块大小
Total PE	总物理区块数
Free PE	可用物理区块数
Allocatable	是否可分配
PV UUID	物理卷唯一标识

vgdisplay 输出字段详解：

字段	说明
VG Name	卷组名称
VG Access	访问模式 (read/write/read-only)
VG Status	状态 (resizable/resizeable)
MAX LV	最大逻辑卷数
Cur LV	当前逻辑卷数
MAX PV	最大物理卷数
Cur PV	当前物理卷数
VG Size	卷组总大小
PE Size	物理区块大小
Total PE	总物理区块数
Alloc PE / Size	已分配 PE / 大小
Free PE / Size	可用 PE / 大小

lvdisplay 输出字段详解：

字段	说明
LV Name	逻辑卷名称
VG Name	所属卷组
LV UUID	逻辑卷唯一标识
LV Write Access	写访问权限
LV Status	逻辑卷状态

字段	说明
LV Size	逻辑卷大小
Current LE	当前逻辑区块数
Segments	映射的物理区块信息

2.3 创建卷组

vgcreate - 创建卷组：

```
# ***** 卷组管理 *****

# 步骤1: 创建卷组 (Volume Group)
# vgcreate = 创建新的卷组
# vg01 = 卷组名称 (自定义, 建议使用有意义的名称)
# /dev/vdb1 = 要加入卷组的物理卷 (可以有多个PV)
# 卷组是物理卷的集合, 形成一个存储池
vgcreate vg01 /dev/vdb1

# 输出说明:
# Volume group "vg01" successfully created
# 表示卷组创建成功

# 步骤2: 查看卷组详细信息
# vgs = 显示卷组的完整详细信息
# 包括: VG名称、PV数量、LV数量、PE大小、总空间等
vgs

# 步骤3: 简短查看卷组
# vgs = 以表格形式简短显示卷组信息
# 适合快速查看多个 VG 的状态
vgs

# 输出示例:
# VG    #PV #LV #SN Attr   VSize   VFree
# vg01   1   0   0 wz--n- 511.00m 511.00m
# ↑     ↑   ↑   ↑   ↑       ↑         ↑
# 名称 PV数 LV数 快照 属性     总大小   空闲空间
#
# 属性说明 (wz--n-):
# w = 可写 (writable)
# z = 可调整大小 (resizable)
# - = 无此属性
# n = 普通卷组 (normal)
# - = 无此属性
```

创建卷组时指定PE大小:

```
# ***** 创建卷组时指定PE大小 *****

# 创建卷组, 指定PE大小为16MiB
# -s 16M = 设置物理区块(PE)大小为16MiB
# PE = Physical Extent, 是LVM分配空间的最小单位
# PE大小创建后不可更改, 需根据使用场景选择
vgcreate -s 16M vg01 /dev/vdb1

# 参数说明:
# -s, --physicalextentsize 16M = 设置PE大小为16MiB
# 可选单位: K, M, G, T, P (默认为MiB)
# PE大小必须是2的幂次方 (1, 2, 4, 8, 16, 32, 64...)

# 多个PV创建卷组示例
# vgcreate可以同时多个物理卷加入卷组
vgcreate myvg /dev/vdb1 /dev/vdb2 /dev/vdc1
```

vgcreate 完整参数说明:

参数	说明
<code>-s, --physicalextentsize Size</code>	设置物理区块大小 (PE Size), 默认 4MiB
<code>--addtag Tag</code>	添加标签到卷组
<code>--alloc AllocationPolicy</code>	设置分配策略 (normal/contiguous/inherit)
<code>-p, --maxlogicalvolumes MaxLVs</code>	设置最大逻辑卷数
<code>--metadataprofile Profile</code>	使用配置文件配置元数据
<code>--dataalignment Size</code>	覆盖默认数据对齐
<code>--dataalignmentoffset Offset</code>	偏移数据对齐
<code>--vgmetadacopies Copies</code>	设置卷组内的元数据副本数

PE 大小说明:

- PE 是 LVM 最小存储单位
- 默认大小: 4MiB
- 可选大小: 1KiB ~ 16GiB (必须是2的幂)
- 较大的 PE 适合大文件系统
- 较小的 PE 可减少空间浪费

⚠ 易错点：PE 大小设置后不可更改

PE Size 的重要限制：

规则	说明
创建时决定	PE 大小在创建 VG 时指定
不可修改	创建后无法更改 PE 大小
影响分配	所有 LV 的大小必须是 PE 的倍数
默认值	不指定时使用 4MiB

示例影响：

```
# PE=4MiB 时, LV 可以是 5MiB (向上取整到8MiB)
# PE=16MiB 时, LV 只能以 16MiB 为单位分配

# 推荐：小文件系统用 4MiB, 大文件系统用 16MiB+
```

SHELL

设置建议：

- 小文件系统 (< 100GB)：使用默认 4MiB
- 大文件系统 (> 100GB)：使用 16MiB 或更大
- 一旦设置，无法更改！

2.4 创建逻辑卷

lvcreate - 创建逻辑卷：

```
# ***** 逻辑卷创建 *****

# 方法1: 按大小创建逻辑卷
# lvcreate = 创建新的逻辑卷
# -L 300M = 指定逻辑卷大小为300MiB
# -n data = 指定逻辑卷名称为 data
# vg01 = 从哪个卷组分配空间
lvcreate -L 300M -n data vg01

# 输出说明:
# Logical volume "data" created
# 表示逻辑卷创建成功

# 方法2: 按扩展块数量创建
# -l 50 = 分配50个LE (逻辑扩展块)
# 1个LE = 1个PE (物理扩展块)
# 如果PE=4MiB, 则50个LE = 200MiB
lvcreate -l 50 -n data vg01

# 方法3: 按百分比创建
# -l 50%VG = 使用卷组50%的空间
# -l 80%FREE = 使用卷组80%的空闲空间
# -l 100%FREE = 使用卷组全部空闲空间
lvcreate -l 50%VG -n data vg01

# 步骤: 查看逻辑卷详细信息
# lvs = 显示逻辑卷的完整详细信息
lvs

# 步骤: 简短查看逻辑卷
# lvs = 以表格形式简短显示逻辑卷信息
lvs

# 输出示例:
# LV      VG      Attr          LSize   Origin Snap%   Move Log Copy%
# Convert
# data vg01 -wi-a----- 300.00M
# ↑      ↑      ↑              ↑       ↑       ↑       ↑       ↑
# 名称 卷组 属性          大小    快照源 快照%   移动日志 转换进度
#
# 属性说明 (-wi-a-----):
# - = 无此属性
# w = 可写 (writable)
# i = 继承分配策略 (inherited alloc policy)
# - = 无此属性
```

```
# a = 活动状态 (active, 已挂载或可用)
# - = 无此属性
```

参数说明:

SHELL

```
# ***** lvcreate 参数详解 *****

# 基本语法:
# lvcreate [选项] -n 逻辑卷名称 卷组名称

# 大小指定方式 (三选一) :
-L 300M           # 指定绝对大小: 300MiB
-L +200M          # 在现有基础上增加200MiB (扩展时使用)
-l 50             # 指定LE数量: 50个逻辑扩展块
-l 50%VG          # 使用卷组50%的总空间
-l 80%FREE        # 使用卷组80%的空闲空间

# 逻辑卷名称 (必需) :
-n mylv           # 指定逻辑卷名称
# 命名规范: 使用小写字母、数字、下划线、连字符
# 示例: data, database_lv, web-storage

# 其他常用选项:
--type linear     # 指定LV类型 (默认linear, 支持striped/mirror/raid)
--stripes 2       # 创建条带卷 (跨2个PV分布数据)
--mirrors 1       # 创建镜像卷 (1个镜像副本)
```

lvcreate 完整选项表:

选项	说明	示例
-L, --size Size	指定逻辑卷大小	-L 500M
-l, --extents Number[%{VG\ PVS\ FREE}]	指定扩展块数量或百分比	-l 50%VG
-n, --name Name	指定逻辑卷名称	-n mylv
-V, --virtualsize Size	创建稀疏逻辑卷	-V 1T
-m, --mirrors Number	创建镜像卷	-m 1
--mirrors +1	添加镜像	
--type linear	striped	mirror
-i, --stripes Number	条带数量	-i 2
-I, --stripesize Size	条带大小	-I 64

选项	说明	示例
<code>--alloc contiguous</code>	cling	normal`
<code>-Z, --zero y\ n</code>	是否填充零	<code>-Z n</code>
<code>--nosync</code>	不同步镜像	

大小单位说明：

单位后缀	大小	说明
<code>b</code>	Bytes	字节
<code>s</code>	Sectors	扇区 (512字节)
<code>k</code>	Kilobytes	千字节
<code>m</code>	Megabytes	兆字节
<code>g</code>	Gigabytes	吉字节
<code>t</code>	Terabytes	太字节
<code>p</code>	Petabytes	拍字节
<code>e</code>	Exabytes	艾字节

逻辑卷路径：

- `/dev/vgname/lvname`
- `/dev/mapper/vgname-lvname`

2.5 格式化和挂载

创建文件系统并挂载：

```
# ***** 文件系统创建与挂载 *****

# 步骤1: 创建XFS文件系统
# mkfs.xfs = 在逻辑卷上创建XFS文件系统
# /dev/vg01/data = 逻辑卷设备路径
# 也可以使用路径: /dev/mapper/vg01-data
mkfs.xfs /dev/vg01/data

# 输出说明:
# meta-data=/dev/vg01/data isize=512 ...
# data      = bsize=4096 blocks=76800 ...
# ...
# 表示文件系统创建成功

# 为什么选择XFS?
# XFS = RHEL9默认文件系统
# 优势: 高性能、支持大文件、可在线扩展
# 注意: XFS只能扩展, 不能缩减!

# 如果需要EXT4文件系统(可缩减):
# mkfs.ext4 /dev/vg01/data

# 步骤2: 创建挂载点目录
# mkdir = 创建目录
# /mnt/data = 挂载点路径(自定义, 通常在/mnt下)
# 挂载点是一个空目录, 用于访问文件系统内容
mkdir /mnt/data

# 步骤3: 挂载文件系统
# mount = 将文件系统附加到目录树
# /dev/vg01/data = 逻辑卷设备
# /mnt/data = 挂载点目录
mount /dev/vg01/data /mnt/data

# 步骤4: 验证挂载
# df -h = 显示已挂载文件系统及磁盘使用情况(人类可读格式)
# grep data = 过滤包含"data"的行
df -h | grep data

# 输出示例:
# Filesystem                Size  Used Avail Use% Mounted on
# /dev/mapper/vg01-data    297M   5.6M  291M   2% /mnt/data
# ↓                        ↓      ↓      ↓      ↓
# 设备路径                总大小 已用 可用 使用率 挂载点
#
```

```
# 说明：
# - 设备显示为 /dev/mapper/vg01-data (设备映射器路径)
# - 实际大小约为300M, 其中约3M用于元数据
# - 使用率仅2%, 因为刚创建的空文件系统

# 步骤5: 验证文件系统类型
# blkid = 显示块设备属性 (文件系统类型、UUID等)
blkid /dev/vg01/data

# 输出示例:
# /dev/mapper/vg01-data: UUID="xxx" TYPE="xfs" ...
# ↑                ↑                ↑
# 设备              唯一标识符      文件系统类型

# 步骤6: 测试写入
# 在挂载点创建测试文件
echo "Hello LVM" > /mnt/data/test.txt
cat /mnt/data/test.txt
```

持久化挂载 (推荐) :

```
# ***** 持久化挂载配置 *****

# 临时挂载在重启后会失效
# 要实现开机自动挂载，需要编辑 /etc/fstab

# 步骤1：获取设备UUID
# blkid = 显示块设备的UUID和文件系统类型
# UUID = 唯一标识符，比设备名更稳定 (/dev/sdX可能变化)
blkid /dev/vg01/data

# 输出示例：
# /dev/mapper/vg01-data: UUID="550e8400-e29b-41d4-a716-446655440000" TYPE="xfs"

# 步骤2：编辑fstab配置文件
vim /etc/fstab

# 添加以下行（使用UUID）：
# UUID=550e8400-e29b-41d4-a716-446655440000 /mnt/data xfs
# defaults 0 0
# ↑ ↑ ↑ ↑
# ↑ ↑
# 设备标识 (UUID) 挂载点 类型 选项
# dump fsck
#
# 字段说明：
# 第1字段：设备 (UUID、设备路径、LABEL)
# 第2字段：挂载点目录
# 第3字段：文件系统类型 (xfs, ext4, nfs等)
# 第4字段：挂载选项 (defaults包含rw, suid, dev, exec, auto, nouser,
# async)
# 第5字段：dump备份 (0=不备份, 1=每天备份)
# 第6字段：fsck检查顺序 (0=不检查, 1=根分区, 2=其他分区)

# 或使用设备路径（不推荐，设备可能变化）：
# /dev/vg01/data /mnt/data xfs defaults 0 0

# 步骤3：测试fstab配置
# mount -a = 挂载fstab中所有未挂载的文件系统
# 如果配置有误，会报错但不会损坏系统
mount -a

# 步骤4：验证挂载
df -h | grep data
```

第三部分：管理LVM逻辑卷

3.1 扩展卷组

添加新物理卷到卷组：

```
# ***** 扩展卷组 *****

# 场景：卷组空间不足，需要添加新磁盘
# 步骤1：先对新磁盘/分区进行分区（如果需要）
# 确保分区类型为 LVM
parted /dev/vdb mkpart primary 1MiB 512MiB
parted /dev/vdb set 2 lvm on

# 步骤2：创建新的物理卷
# pvcreate = 将新分区初始化为物理卷
# /dev/vdb2 = 新的分区（假设第二个分区）
pvcreate /dev/vdb2

# 步骤3：将物理卷添加到现有卷组
# vgextend = 扩展卷组，添加新的物理卷
# vg01 = 要扩展的卷组名称
# /dev/vdb2 = 要添加的物理卷
# 可以一次添加多个PV：vgextend vg01 /dev/vdb2 /dev/vdc1
vgextend vg01 /dev/vdb2

# 输出说明：
# Volume group "vg01" successfully extended
# 表示卷组扩展成功

# 步骤4：验证扩展结果
# vgdisplay = 显示卷组详细信息
vgdisplay vg01

# 输出示例：
# VG Size                1020.00 MiB
# Free  PE / Size        254 / 1016.00 MiB
# ↑      ↑      ↑        ↑      ↑
# 空闲   物理扩展块数量   物理扩展块大小
#
# 说明：
# - VG Size 从 511MiB 增加到 1020MiB
# - 增加了约509MiB可用空间
# - 1016MiB 可用空间（部分用于元数据）

# 步骤5：简短查看卷组状态
vgs

# 输出示例：
# VG    #PV #LV #SN Attr   VSize    VFree
# vg01   2   1   0 wz--n- 1020.00m 1016.00m
```

↑ ↑ ↑ ↑ ↑
名称 PV数 (从1变2) 总大小 空闲空间

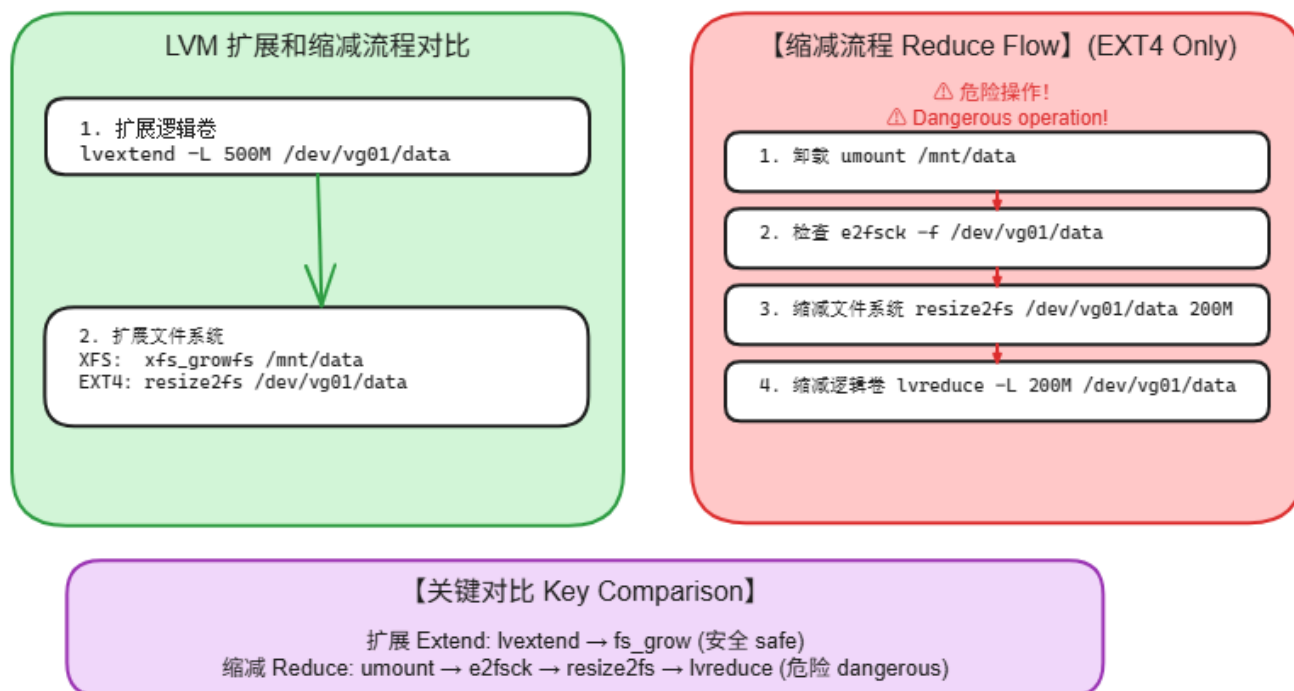
扩展卷组后，可以创建新的逻辑卷或扩展现有逻辑卷

3.2 扩展逻辑卷

LVM扩展和缩减流程对比：

LVM 扩展和缩减流程对比 LVM Extend vs Reduce Flow Comparison

⚠ XFS 只能扩展，不能缩减！
⚠ XFS can only extend, not reduce!



LVM vs VG 关系说明：

***** LVM层级关系说明 *****

LV (Logical Volume) = 从 VG 中分配空间的逻辑卷

VG (Volume Group) = 包含多个 PV 的存储池

PV (Physical Volume) = 实际的物理分区/磁盘

层级关系:

磁盘分区 (PV) → 卷组 (VG) → 逻辑卷 (LV) → 文件系统 → 挂载点

关键点:

1. LV 创建在 VG 之上
2. LV 大小受 VG 可用空间限制
3. 一个 VG 可以包含多个 LV
4. VG 提供空间池, LV 从中分配
5. LV 可以动态调整大小 (VG 空间允许时)
6. LV 可以跨多个 PV 分配空间

扩展逻辑卷和文件系统:

```
# ***** 扩展逻辑卷 *****

# 步骤1: 查看卷组可用空间
# 确保有足够的空闲空间用于扩展
# vgdisplay | grep Free = 过滤显示空闲空间信息
vgdisplay vg01 | grep Free

# 输出示例:
# Free  PE / Size   50 / 200.00 MiB
# ↑      ↑   ↑      ↑   ↑
# 空闲   PE数 大小   可用空间

# 方法1: 扩展逻辑卷到指定大小
# lvextend = 扩展逻辑卷命令
# -L 500M = 设置新大小为500MiB (绝对大小)
# /dev/vg01/data = 目标逻辑卷
lvextend -L 500M /dev/vg01/data

# 输出说明:
# Size of logical volume vg01/data changed from 300.00 MiB (75
# extents) to 500.00 MiB (125 extents).
# Logical volume vg01/data successfully resized.
# ↑                                     ↑
# 旧大小 (75个PE)                       新大小 (125个PE)

# 方法2: 在现有基础上增加指定大小
# -L +200M = 在当前大小基础上增加200MiB
# +号表示增量扩展
lvextend -L +200M /dev/vg01/data

# 方法3: 按百分比扩展
# -l +50%FREE = 使用卷组50%的空闲空间
# -l 100%FREE = 使用全部空闲空间
# -l = 按扩展块数量分配 (-L按大小分配)
lvextend -l +50%FREE /dev/vg01/data

# 方法4: 自动扩展文件系统 (推荐!)
# -r = --resizefs, 自动扩展文件系统
# 此选项会自动调用正确的扩展命令:
#   - XFS: 调用 xfs_growfs
#   - EXT4: 调用 resize2fs
# 一条命令完成LV和文件系统的扩展
lvextend -r -L 500M /dev/vg01/data

# 输出说明:
```

```
# Size of logical volume vg01/data changed from 300.00 MiB ...  
# [XXX] xx%  
# Logical volume vg01/data successfully resized.  
# ↑  
# xfs_growfs 或 resize2fs 的进度显示
```

扩展文件系统（如果未使用 **-r** 选项）：

```
# ***** 扩展文件系统 *****

# 重要：扩展LV后，必须扩展文件系统才能使用新增空间！

# XFS文件系统扩展
# xfs_growfs = 扩展XFS文件系统
# /mnt/data = 挂载点目录（注意：是挂载点，不是设备路径）
# XFS必须在挂载状态下扩展
xfs_growfs /mnt/data

# 输出示例：
# data blocks changed from 76800 to 128000
# ↑                               ↑
# 旧块数                         新块数

# EXT4文件系统扩展
# resize2fs = 扩展EXT4文件系统
# /dev/vg01/data = 设备路径（EXT4使用设备路径）
# EXT4可以在挂载或卸载状态下扩展（推荐挂载状态）
resize2fs /dev/vg01/data

# 输出示例：
# resize2fs 1.46.5 (30-Dec-2021)
# Filesystem at /dev/vg01/data is mounted on /mnt/data; on-line
# resizing required
# old_desc_blocks = 2, new_desc_blocks = 3
# The filesystem on /dev/vg01/data is now 128000 (1k) blocks
# long.

# 验证扩展结果
# df -h = 显示文件系统大小和使用情况
df -h /mnt/data

# 输出示例：
# Filesystem                Size      Used Avail Use% Mounted on
# /dev/mapper/vg01-data 497M    6.0M   491M    2% /mnt/data
# ↑                ↑      ↑      ↑      ↑
# 约500M            总大小 已用   可用   挂载点
```

lvextend 完整选项表：

选项	说明	示例
-L, --size Size[+/-]	设置或调整大小	-L 500M

选项	说明	示例
<code>-l, --extents Number[+/-]% {VG\ PVS\ FREE}</code>	按扩展块数量或百分比	<code>-l +50%FREE</code>
<code>-r, --resizefs</code>	自动扩展文件系统	<code>-r -L 1G</code>
<code>-i, --stripes Number</code>	条带数量	<code>-i 2</code>
<code>-I, --stripesize Size</code>	条带大小	<code>-I 64</code>
<code>--alloc AllocationPolicy</code>	分配策略	<code>--alloc contiguous</code>
<code>--noudevsync</code>	禁用 udev 同步	

扩展逻辑卷和文件系统：

SHELL

```
# 查看卷组可用空间
vgdisplay vg01 | grep Free

# 扩展逻辑卷到500M
lvextend -L 500M /dev/vg01/data

# 或增加200M
lvextend -L +200M /dev/vg01/data

# 或使用卷组50%的空闲空间
lvextend -l +50%FREE /dev/vg01/data

# 自动扩展文件系统（推荐）
lvextend -r -L 500M /dev/vg01/data

# 输出示例：
# Size of logical volume vg01/data changed from 300.00 MiB (75
# extents) to 500.00 MiB (125 extents).
# Logical volume vg01/data successfully resized.
```

扩展文件系统：

```
# XFS文件系统扩展
xfs_growfs /mnt/data

# EXT4文件系统扩展
resize2fs /dev/vg01/data

# 输出示例:
# data blocks changed from 76800 to 128000
```

xfs_growfs vs resize2fs 详解:

特性	xfs_growfs (XFS)	resize2fs (EXT4)
操作对象	挂载点目录	设备路径
扩容	✔ 支持	✔ 支持
缩容	✘ 不支持	✔ 支持
是否需要卸载	✘ 可以在线操作	⚠ 缩容时需要
命令示例	xfs_growfs /mnt/data	resize2fs /dev/vg01/lv
自动扩展	lvextend -r 会自动调用	lvextend -r 会自动调用

xfs_growfs 常用选项:

选项	说明
-d	扩展到最大可用空间 (默认)
-s Size	指定扩展大小
-n	仅显示新大小, 不实际修改

resize2fs 常用选项:

选项	说明
-p	显示进度条
-d	调试模式
-f	强制执行
Size	指定新大小 (可选)

注意:

- XFS只能扩容，不能缩容
- 使用 `xfs_growfs` 挂载点 扩展XFS
- 使用 `resize2fs` 设备 扩展EXT4
- 使用 `-r` 选项可以自动扩展文件系统: `lvextend -r -L 500M /dev/vg01/data`

⚠ 易错点: XFS 无法缩容

文件系统缩容限制:

文件系统	扩容	缩容	说明
XFS	✅ 支持	❌ 不支持	RHEL9 默认, 只能变大
EXT4	✅ 支持	✅ 支持	可以缩容, 但有风险

为什么 XFS 不能缩容?

- XFS 设计为高性能文件系统
- 缩容需要重新分配所有数据块
- 技术复杂且风险极高

操作建议:

SHELL

```
# 扩容 XFS (安全)
lvextend -L 1G /dev/vg01/data
xfs_growfs /mnt/data

# 需要缩小空间时的替代方案:
# 1. 备份数据
# 2. 删除 LV, 重新创建更小的
# 3. 格式化, 恢复数据
```

记忆技巧: XFS = eXpandable File System (只可扩展)

3.3 缩减逻辑卷

⚠ 危险操作: 缩减逻辑卷

***** 重要警告 *****

缩减逻辑卷是 LVM 最危险的操作之一！

风险：

- 数据丢失（如果顺序错误）
- 文件系统损坏（如果缩减失败）
- 无法恢复（除非有完整备份）

限制：

- XFS 文件系统不支持缩减！
- 只有 EXT4 支持缩减
- 必须按正确顺序执行

缩减逻辑卷是 LVM 最危险的操作之一，必须按正确顺序执行：

```
# ***** 缩减逻辑卷 (EXT4 仅限) *****

# ⚠ 警告：此操作仅适用于 EXT4 文件系统
# ⚠ XFS 不支持缩减！

# 步骤1：卸载文件系统（必须！）
# umount = 卸载文件系统
# 必须先卸载才能安全地缩减文件系统
# 如果不卸载，数据可能损坏
umount /mnt/data

# 步骤2：检查文件系统
# e2fsck = EXT4文件系统检查工具
# -f = 强制检查（即使标记为clean）
# 确保文件系统没有错误才能进行缩减
e2fsck -f /dev/vg01/data

# 输出示例：
# Pass 1: Checking inodes, blocks, and sizes
# Pass 2: Checking directory structure
# Pass 3: Checking directory connectivity
# Pass 4: Checking reference counts
# Pass 5: Checking group summary information
# data: 11/128016 files (0.0% non-contiguous), 50760/512000
# blocks
# ↑                ↑                ↑
# 卷名                文件数          块数
# 状态正常，可以继续

# 步骤3：缩减文件系统（必须先于LV！）
# resize2fs = 缩减/扩展EXT4文件系统
# /dev/vg01/data = 设备路径
# 200M = 新的目标大小
# ⚠ 关键：先缩减文件系统，再缩减逻辑卷
# ⚠ 顺序错误会导致数据丢失！
resize2fs /dev/vg01/data 200M

# 输出示例：
# resize2fs 1.46.5 (30-Dec-2021)
# Resizing the filesystem on /dev/vg01/data to 204800 (1k)
# blocks.
# The filesystem on /dev/vg01/data is now 204800 blocks long.

# 步骤4：缩减逻辑卷
# lvreduce = 缩减逻辑卷命令
```

```
# -L 200M = 设置新大小为200MiB
# ⚠️ LV的大小必须 >= 文件系统大小
# ⚠️ 如果LV < 文件系统, 数据会被截断!
lvreduce -L 200M /dev/vg01/data

# 输出示例:
# WARNING: Reducing active logical volume to 200.00 MiB
# THIS MAY DESTROY YOUR DATA (filesystem etc.)
# ⚠️ 警告提示, 确认后输入 y 继续
# Do you really want to reduce vg01/data? [y/n]: y
# Size of logical volume vg01/data changed from 300.00 MiB (75
# extents) to 200.00 MiB (50 extents).
# Logical volume vg01/data successfully resized.

# 步骤5: 重新挂载
# mount = 挂载文件系统
# 挂载后检查数据完整性
mount /dev/vg01/data /mnt/data

# 步骤6: 验证数据和大小
# df -h = 检查文件系统大小
df -h /mnt/data

# 输出示例:
# Filesystem              Size  Used Avail Use% Mounted on
# /dev/mapper/vg01-data 197M   5.6M  191M   3% /mnt/data
# ↑
# 约200M

# 步骤7: 检查数据完整性
# 列出文件内容, 确保没有丢失
ls -la /mnt/data
cat /mnt/data/test.txt
```

安全记忆口诀:

***** 安全操作顺序 *****

缩减逻辑卷必须按顺序：

- 1. 卸载 (umount)
- 2. 检查 (e2fsck)
- 3. 缩文件系统 (resize2fs) ← 先变FS
- 4. 缩逻辑卷 (lvreduce) ← 后变LV
- 5. 挂载 (mount)

记忆口诀：FS先变小，LV后跟随

⚠ 易错点：缩减逻辑卷的顺序

缩容操作的致命陷阱：

步骤	操作	说明
1	卸载	必须先卸载！
2	检查文件系统	e2fsck 确保数据完整
3	缩减文件系统	先缩减 FS，再缩减 LV
4	缩减逻辑卷	LV 大小 ≥ FS 大小
5	挂载	验证数据完整性

常见错误：

SHELL

```
# X 错误：直接缩减 LV (数据丢失！)
lvreduce -L 200M /dev/vg01/data # 未缩减文件系统

# X 错误：顺序颠倒 (数据损坏！)
lvreduce -L 200M /dev/vg01/data # 先缩减 LV
resize2fs /dev/vg01/data 200M # 后缩减 FS (已损坏)

# ✓ 正确：先缩减文件系统，再缩减 LV
resize2fs /dev/vg01/data 200M # 先缩减 FS
lvreduce -L 200M /dev/vg01/data # 后缩减 LV
```

安全记忆：FS 先变小，LV 后跟随

XFS 缩容：

- 不支持！无法缩减 XFS 文件系统
-

3.3 缩减卷组

移除物理卷：

```
# 将数据从一个PV移动到其他PV
pvmove /dev/vdb2
```

```
# 从卷组移除PV
vgreduce vg01 /dev/vdb2
```

```
# 删除物理卷
pvremove /dev/vdb2
```

SHELL

3.4 删除LVM

删除逻辑卷、卷组和物理卷：

```
# ***** 删除LVM *****

# ⚠ 警告：删除操作不可逆！确保备份重要数据！

# 步骤1：卸载文件系统
# umount = 卸载已挂载的文件系统
# 必须先卸载才能删除逻辑卷
umount /mnt/data

# 验证卸载
# df -h = 确认已不在挂载列表中
df -h | grep data
# 无输出表示已卸载

# 步骤2：编辑fstab删除挂载条目
# vim = 编辑配置文件
# /etc/fstab = 系统挂载配置文件
# 删除或注释掉相关行，避免开机挂载错误
vim /etc/fstab

# 示例：注释掉fstab中的LVM挂载行
# UUID=xxx /mnt/data xfs defaults 0 0
# 改为：
# #UUID=xxx /mnt/data xfs defaults 0 0
# ↑
# 添加#号注释此行

# 步骤3：删除逻辑卷
# lvremove = 删除逻辑卷命令
# /dev/vg01/data = 要删除的逻辑卷
# 系统会提示确认，需要输入 y
lvremove /dev/vg01/data

# 输出说明：
# Do you really want to remove active logical volume vg01/data?
# [y/n]: y
# Logical volume "data" successfully removed
# 表示逻辑卷删除成功

# 或者使用 -f 选项强制删除（跳过确认）
lvremove -f /dev/vg01/data

# 步骤4：删除卷组
# ⚠ 卷组中的所有逻辑卷必须先删除！
# vgreduce = 从卷组移除物理卷
```

```
# vgrename = 删除整个卷组
# 如果VG中有其他LV, 必须先删除
vgremove vg01

# 输出说明:
# Volume group "vg01" successfully removed
# 表示卷组删除成功

# 步骤5: 删除物理卷
# pvremove = 删除物理卷上的LVM标签
# /dev/vdb1 = 要删除的物理卷
# 此操作不会删除分区, 只删除LVM元数据
pvremove /dev/vdb1

# 输出说明:
# Labels on physical volume "/dev/vdb1" successfully wiped
# 表示LVM标签已清除

# 步骤6: 验证删除结果
# pvs = 显示物理卷 (应该没有输出)
pvs
# vgs = 显示卷组 (应该没有输出)
vgs
# lvs = 显示逻辑卷 (应该没有输出)
lvs

# 步骤7: 可选 - 删除分区
# 如果不再需要分区, 可以删除
# parted /dev/vdb rm 1 = 删除分区1
# ⚠ 此操作会删除分区上的所有数据!
parted /dev/vdb print # 先查看分区
parted /dev/vdb rm 1 # 删除分区1
```

删除顺序总结:

***** LVM删除顺序 *****

从上层到下层依次删除：

1. umount 卸载文件系统
2. vim fstab 删除持久化配置
3. lvremove 删除逻辑卷（LV）
4. vgremove 删除卷组（VG）
5. pvremove 删除物理卷（PV）
6. 可选：删除分区

记忆：从上到下，逐层删除

演示：LVM实践

演示目标

掌握 LVM 的创建和管理

演示步骤

步骤1：创建分区

```
# ***** LVM完整实践流程 *****

# 步骤1.1: 查看可用磁盘
# lsblk = 列出块设备信息
# 可以看到所有磁盘和分区
lsblk

# 输出示例:
# NAME      MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
# vdb        252:16   0   512M  0 disk
# └─vdb1     252:17   0   500M  0 part
# ↑
# 磁盘名

# 步骤1.2: 创建分区
# 启动parted交互式工具
parted /dev/vdb

# 在parted提示符下执行以下命令:

# 创建GPT分区表
mklabel gpt

# 创建主分区 (1MiB到512MiB)
mkpart primary 1MiB 512MiB

# 设置分区1为LVM类型
set 1 lvm on

# 退出parted
quit
```

步骤2: 创建LVM

```
# ***** 创建LVM *****

# 步骤2.1: 创建物理卷
# pvcreate = 将分区初始化为物理卷
pvcreate /dev/vdb1

# 步骤2.2: 创建卷组
# vgcreate = 创建卷组, 命名为myvg
# myvg = 自定义的卷组名称
vgcreate myvg /dev/vdb1

# 步骤2.3: 创建逻辑卷
# lvcreate = 创建逻辑卷
# -L 200M = 大小为200MiB
# -n mylv = 命名为mylv
# myvg = 从myvg卷组分配空间
lvcreate -L 200M -n mylv myvg

# 步骤2.4: 格式化为XFS文件系统
# mkfs.xfs = 创建XFS文件系统
mkfs.xfs /dev/myvg/mylv

# 步骤2.5: 创建挂载点
mkdir /data

# 步骤2.6: 挂载逻辑卷
mount /dev/myvg/mylv /data

# 步骤2.7: 验证挂载
df -h /data
```

步骤3: 扩展逻辑卷

```
# ***** 扩展逻辑卷 *****

# 步骤3.1: 查看当前逻辑卷大小
# lvs = 显示逻辑卷简短信息
lvs

# 输出示例:
# LV   VG   Attr           LSize   Pool Origin Data%   Meta%   Move Log
# Cpy%Sync Convert
# mylv myvg -wi-a----- 200.00M
# ↑     ↑     ↑           ↑
# 名称 卷组 属性           当前大小

# 步骤3.2: 扩展逻辑卷到300M
# lvextend = 扩展逻辑卷
# -L 300M = 新大小为300MiB (绝对值)
# /dev/myvg/mylv = 目标逻辑卷
lvextend -L 300M /dev/myvg/mylv

# 输出示例:
# Size of logical volume myvg/mylv changed from 200.00 MiB (50
# extents) to 300.00 MiB (75 extents).

# 步骤3.3: 扩展XFS文件系统
# xfs_growfs = 扩展XFS文件系统
# /data = 挂载点目录 (不是设备路径)
xfs_growfs /data

# 步骤3.4: 验证扩展结果
# df -h = 显示文件系统大小和使用情况
df -h /data

# 输出示例:
# Filesystem              Size  Used Avail Use% Mounted on
# /dev/mapper/myvg-mylv 297M   5.6M  291M   2% /data
# ↑                        ↑
# 约300M (扩展成功)

# 使用推荐的 -r 选项可以一步完成:
# lvextend -r -L 300M /dev/myvg/mylv
```

相关练习 (LVM逻辑卷管理)

练习名称	对应章节	说明
练习: Create and Extend Logical Volumes	第08章 P252	创建和扩展逻辑卷
练习: Managing Logical Volumes	第08章 P272	管理逻辑卷 (扩展/缩减/删除)

下午 - NFS网络存储

课前故事：网络存储的诞生

为什么需要网络存储？

随着企业业务发展，单一服务器的存储能力已无法满足需求：

场景 1：多台服务器需要访问同一份数据

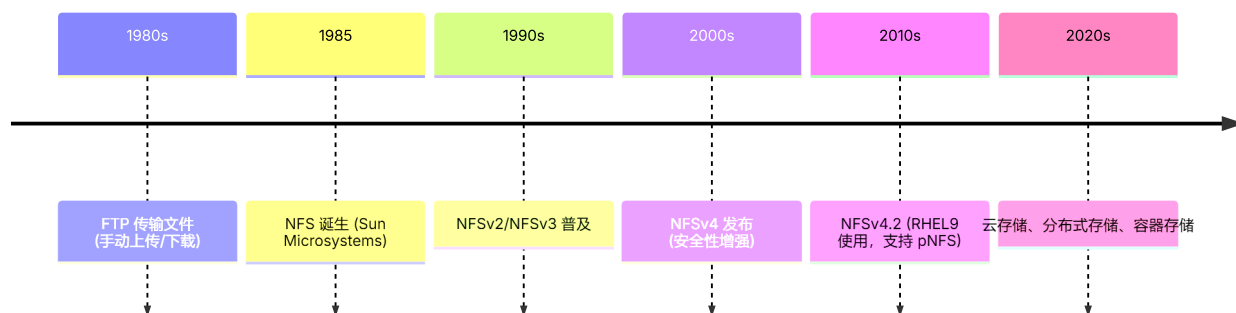
- 传统方案：每台服务器存储一份数据副本 → 数据不一致！
- 网络存储方案：集中存储，多台服务器共享 → 数据一致！

场景 2：服务器存储空间不足

- 传统方案：停机、添加硬盘、重新分区、恢复数据 → 耗时且风险高
- 网络存储方案：远程挂载存储池 → 在线扩展！

网络存储技术演进历史：

网络存储技术发展史



NFS 的革命性设计：

NFS 让远程存储"像本地一样"：

传统文件共享：

用户 → FTP上传 → 服务器A → 服务器B下载 → 用户修改 → FTP上传
↑ 每次都要传输完整文件

NFS 网络存储：

用户 — 直接访问 → NFS 服务器
↑ ↑
透明操作 集中管理
即时生效 数据一致

NFS 在现代架构中的应用：

- **Web 集群：**多台 Web 服务器共享静态资源
- **数据共享：**多台应用服务器访问同一数据目录
- **备份集中：**统一备份数据到 NFS 服务器
- **用户目录：**集中管理用户家目录

第四部分：NFS基础

为什么需要学习 NFS？

通过课前故事，我们了解了 NFS 如何解决多服务器共享数据的难题。现在让我们深入学习 NFS 的配置和使用。

本部分将学习：

- NFS 的工作原理和版本差异
- NFS 服务端配置 (/etc/exports)
- NFS 客户端挂载和故障排查
- autofs 自动挂载配置

掌握这些知识，你将能够：

- 搭建网络存储服务
- 实现多服务器数据共享
- 配置自动挂载简化管理

4.1 NFS 介绍

NFS客户端-服务器架构：

NFS 客户端-服务器架构

FS Client-Server Architecture

【NFS服务器端 NFS Server】



NFSv4 协议
通过网络传输
TCP 端口 2049

【NFS客户端 Client】



NFSv3 (旧版本)

需要 `rpcbind`
`showmount -e` 可用

NFSv4 (推荐)

不需要 `rpcbind`
仅固定端口 2049

NFS (Network File System):

网络文件系统，允许远程主机通过网络访问本地文件系统

- 支持标准的Linux权限
- 透明访问，像本地文件系统一样
- RHEL9 使用 NFSv4.2
- 仅支持 TCP 协议

NFSv4 vs NFSv3 详解:

特性	NFSv4	NFSv3
协议	仅 TCP	TCP 或 UDP
RPC	不需要 rpcbind	需要 rpcbind
端口	固定端口 2049	动态端口
安全性	支持 RPCSEC_GSS, Kerberos	弱认证
锁机制	内置	需要 rpc.lockd
showmount	✗ 不适用	✓ 可用
查询方法	手动挂载根目录	showmount -e

showmount 命令详解:

SHELL

```
# 查询 NFSv3 服务器导出
showmount -e server

# 输出示例 (仅 NFSv3) :
# Export list for server:
# /shares/operation 192.168.0.0/24
# /shares/production 192.168.0.0/24

# NFSv4 服务器查询方法
# 方法1: 手动挂载根目录查看
mount server:/ /mnt
ls /mnt

# 方法2: SSH 到服务器查看
ssh server exportfs -v

# 方法3: 查看导出配置
ssh server cat /etc/exports.d/*.exports
```

安装 NFS 客户端:

```
# 安装 nfs-utils
dnf install nfs-utils

# 查看NFS版本
nfsstat -s

# 输出示例:
# Server rpc stats:
# ...version 4: (...)
```

4.2 挂载 NFS 共享

挂载步骤:

1. 识别 NFS 共享
2. 创建挂载点
3. 挂载文件系统

临时挂载:

```
# 创建挂载点
mkdir /mnt/nfs

# 挂载NFS共享
mount server:/share /mnt/nfs

# 参数说明:
# server = NFS服务器主机名或IP
# /share = 导出的目录
# /mnt/nfs = 本地挂载点

# 查看挂载
df -h | grep nfs
```

持久挂载 (/etc/fstab) :

```
# 编辑 fstab
vim /etc/fstab

# 添加以下内容:
server:/share /mnt/nfs nfs defaults 0 0

# 立即测试所有/etc/fstab中的自动挂载
mount -a

# 立即测试/etc/fstab中的/mnt/nfs自动挂载
mount /mnt/nfs

# 验证
df -h | grep nfs
```

fstab 字段说明:

字段	说明
第1字段	NFS共享 (host:/path)
第2字段	挂载点目录
第3字段	文件系统类型 (nfs 或 nfs4)
第4字段	挂载选项
第5字段	dump备份
第6字段	fsck检查

4.3 挂载选项

NFS 挂载选项完整表:

选项	说明	默认值
基本选项		
rw	读写挂载	✓
ro	只读挂载	✗
suid	允许 setuid	✓
nosuid	禁止 setuid	✗
dev	允许设备文件	✓
nodev	禁止设备文件	✗

选项	说明	默认值
<code>exec</code>	允许执行	✓
<code>noexec</code>	禁止执行	✗
性能选项		
<code>sync</code>	同步写入	✗
<code>async</code>	异步写入	✓
<code>atime</code>	更新访问时间	✓
<code>noatime</code>	不更新访问时间	✗
错误处理		
<code>soft</code>	超时返回错误	✗
<code>hard</code>	持续重试	✓
<code>intr</code>	允许中断	✗
<code>nointr</code>	不允许中断	✓
超时选项		
<code>timeo=n</code>	超时时间(1/10秒)	60 (600)
<code>retrans=n</code>	重试次数	2
大小选项		
<code>rsize=n</code>	读块大小	1048576
<code>wsiz=n</code>	写块大小	1048576
其他选项		
<code>fg</code>	前台挂载	✓
<code>bg</code>	后台挂载	✗
<code>nofail</code>	启动失败不报错	✗

常用挂载选项组合：

默认选项

defaults

高性能异步写入

async,rw,noatime

安全同步写入

sync, rw

数据共享（推荐用于多客户端）

rw, sync, noatime

只读挂载

ro, sync

练习名称	对应章节	说明
练习：MANAGING NETWORK-ATTACHED STORAGE WITH NFS	第09章 P283	使用NFS管理网络附加存储

第五部分：autofs自动挂载

5.1 autofs 介绍

什么是 **autofs**：

自动挂载服务，按需挂载网络文件系统

- 访问时自动挂载
- 不使用时自动卸载
- 无需 root 权限
- 客户端配置，无需服务器配置

安装和启用：

```
# 安装 autofs
dnf install autofs

# 启动并启用服务
systemctl enable --now autofs

# 查看状态
systemctl status autofs
```

5.2 autofs 配置结构

autofs映射方式对比:

autofs 间接映射 vs 直接映射对比 autofs Indirect vs Direct Mapping Comparison

【间接映射 Indirect Mapping】

配置文件

```
/etc/auto.master.d/test.autofs:
/test /etc/auto.test

/etc/auto.test:
data -rw server:/exports/data
```

工作原理

访问 **/test/data**

autofs自动挂载

server:/exports/data → /test/data

超时后自动卸载

- 特点
- 需要基础目录(/test)
 - 目录相对路径(data)
 - 统一管理多个挂载点

【通配符映射 Wildcard Mapping】

```
/etc/auto.wildcard:
* -rw server:/exports/&

* 匹配任意目录名 & 引用匹配的目录名
```

访问/base/data → 挂载server:/exports/data

【直接映射 Direct Mapping】

配置文件

```
/etc/auto.master.d/direct.autofs:
/- /etc/auto.direct

/etc/auto.direct:
/mnt/share -rw server:/exports/share
```

访问 **/mnt/share**

autofs直接挂载

server:/exports/share → /mnt/share

- 特点
- 使用/-作为主挂载点
 - 使用绝对路径(/mnt/share)
 - 每个挂载点单独配置

配置文件:

文件	作用
<code>/etc/auto.master</code>	主映射文件
<code>/etc/auto.master.d/*.autofs</code>	自定义主映射文件
<code>/etc/auto.xxx</code>	直接映射文件

autofs 特点:

- 无需 root 特权运行 mount/umount
- 配置的共享可供所有用户使用
- 不需要永久连接, 可释放资源
- 客户端配置, 无需服务器配置
- 与 mount 命令使用相同选项
- 支持直接和间接映射
- 默认网络文件系统是 NFS

两种映射方式对比:

特性	间接映射	直接映射
基础目录	指定基础路径	<code>/-</code>
挂载点	相对于基础目录	绝对路径
目录创建	自动创建基础目录, 子目录只有访问时才会挂载远程文件系统	自动创建完整路径
配置复杂度	较简单	稍复杂
适用场景	适合同一个基础目录下创建多个相关挂载到子目录, 可以和通配符映射一起使用减少配置行数	适合分散的独立挂载位于不同位置的共享目录

映射文件格式:

```
# 间接映射格式
key [-options] location

# 直接映射格式
/full/path [-options] location
```

5.3 间接映射

配置间接映射：

```
# 1. 创建主映射文件
vim /etc/auto.master.d/demo.autofs

# 添加内容：
/mnt/nfs /etc/auto.nfs

# 参数说明：
# /mnt/nfs = 基础目录
# /etc/auto.nfs = 映射文件
```

SHELL

```
# 2. 创建映射文件
vim /etc/auto.nfs

# 添加内容：
# 只有当用户访问时才由autofs在基础目录下创建实际挂载点 /mnt/nfs/data
data -rw,sync server:/share/data

# 参数说明：
# data = 子目录名 (key)
# -rw,sync = 挂载选项
# server:/share/data = NFS共享
# 只有实际访问(用ls,cd等命令) /mnt/nfs/data 时, 才会自动按需创建挂载点
/mnt/nfs/data
# 过一段时间不再访问则自动卸载挂载点
```

SHELL

```
# 3. 重启 autofs
# umount /mnt/nfs/dat 避免挂载失败
systemctl restart autofs

# 4. 测试
cd /mnt/nfs/data
```

SHELL

5.4 直接映射

配置直接映射：

```
# 1. 创建主映射文件
vim /etc/auto.master.d/direct.autofs

# 添加内容：
/- /etc/auto.direct

# 参数说明：
# /- = 直接映射标志
```

SHELL

```
# 2. 创建映射文件
vim /etc/auto.direct

# 添加内容：
/mnt/data -rw,sync server:/share/data

# 参数说明：
# /mnt/data = 完整本地挂载点
```

SHELL

```
# 3. 重启 autofs
# umount /mnt/nfs/dat 避免挂载失败
systemctl restart autofs
```

SHELL

5.5 通配符映射

使用通配符：

```
# 主映射文件使用相对映射方式
# 配置关联映射文件
vim /etc/auto.nfs

# 使用通配符:
* -rw,sync,fstype=nfs4 server:/shares/&

# 参数说明:
# * = 匹配任意目录名
# & = 被匹配的目录名
```

通配符映射详解:

符号	含义	说明
*	匹配任意目录名	key 字段使用通配符
&	引用匹配的目录名	location 字段中的占位符

通配符映射示例:

```
# auto.indirect 文件内容
* -fstype=nfs4,rw,sync server:/shares/&

# 效果:
# 访问 /base/data      → 挂载 server:/shares/data
# 访问 /base/docs      → 挂载 server:/shares/docs
# 访问 /base/projects → 挂载 server:/shares/projects
```

注意事项:

- 主映射文件不能是直接映射方式(\-)
- 通配符 * 匹配任意子目录名, & 会被替换为实际访问的目录名
- 在服务器导出多个子目录的场景可以减少配置行数, 简化管理
- 如果autofs挂载失败可以尝试以下办法解决:
 - 检查映射配置是否正确
 - 卸载远程挂载点
 - 重启autofs服务

5.6 systemd.automount 方法

systemd.automount 介绍：

除了 autofs 服务，systemd 也提供自动挂载功能

特性	autofs	systemd.automount
配置方式	独立配置文件	/etc/fstab
映射类型	直接+间接	仅直接映射
复杂度	较复杂	较简单
灵活性	高	低

使用方法：

SHELL

```
# 1. 在 /etc/fstab 中添加条目
vim /etc/fstab

# 添加 x-systemd.automount 选项
server:/shares/docs /mnt/docs nfs defaults,x-systemd.automount 0

# 2. 重新加载 systemd
systemctl daemon-reload

# 3. 启动 automount 单元
# unit 命名格式: mnt-docs.automount (将 / 替换为 -)
systemctl start mnt-docs.automount

# 4. 设置开机自启
systemctl enable mnt-docs.automount
```

systemd.automount 选项：

选项	说明
x-systemd.automount	启用自动挂载
x-systemd.idle-timeout=sec	空闲超时时间
x-systemd.device-timeout=sec	设备超时时间
nofail	设备不存在时不报错

注意：

- unit 文件名基于挂载点生成
 - 将路径中的 `/` 替换为 `-`
 - 例如： `/remote/data` → `remote-data.automount`
 - 仅支持直接映射
-

演示：NFS和autofs实践

演示目标

掌握 NFS 挂载和 autofs 配置

演示步骤

步骤1：手动挂载NFS

```

# ***** NFS和autofs实践流程 *****

# 步骤1.1: 安装 nfs-utils 软件包
# dnf = DNF包管理器 (RHEL9默认)
# install = 安装软件包
# nfs-utils = NFS工具包 (包含客户端和服务端工具)
# -y = 自动确认安装 (不需要手动输入y)
dnf install nfs-utils -y

# 输出说明:
# Complete! 表示安装成功

# 步骤1.2: 创建挂载点目录
# mkdir = 创建目录
# /mnt/share = 挂载点路径 (自定义)
mkdir /mnt/share

# 步骤1.3: 挂载NFS共享
# mount = 挂载文件系统
# serverb = NFS服务器主机名 (也可以使用IP地址)
# /shares/operation = 服务器导出的目录路径
# /mnt/share = 本地挂载点目录
mount serverb:/shares/operation /mnt/share

# 参数说明:
# serverb:/shares/operation = NFS共享资源
# 格式: 主机名或IP:/导出路径

# 步骤1.4: 查看NFS共享内容
# ls = 列出目录内容
ls /mnt/share

# 步骤1.5: 验证挂载
# df -h = 显示文件系统及挂载信息
# 人类可读格式 (-h)
df -h | grep serverb

# 预期输出:
# serverb:/shares/operation 1.0G 100M 900M 10% /mnt/share
# ↑                ↑      ↑      ↑      ↑
# NFS资源          总大小 已用 可用 使用率 挂载点

# 步骤1.6: 卸载NFS共享
# umount = 卸载文件系统
# /mnt/share = 要卸载的挂载点

```

```
umount /mnt/share
```

```
# 或使用设备路径卸载:
```

```
# umount serverb:/shares/operation
```

步骤2: 配置autofs

```

# ***** autofs自动挂载配置 *****

# 步骤2.1: 安装 autofs 软件包
# autofs = 自动挂载服务
# -y = 自动确认安装
dnf install autofs -y

# 步骤2.2: 创建主映射文件
# /etc/auto.master.d/ = 主映射文件目录 (推荐位置)
# shares.autofs = 自定义的主映射文件名 (.autofs后缀)
# 使用 vim 编辑器创建文件
vim /etc/auto.master.d/shares.autofs

# 添加以下内容:
# /remote /etc/auto.shares
# ↑      ↑
# 基础目录 映射文件路径
#
# 说明:
# /remote = 基础挂载目录 (autofs会自动创建)
# /etc/auto.shares = 包含子挂载点配置的映射文件

# 步骤2.3: 创建映射文件
# /etc/auto.shares = 映射文件 (文件名与上面配置对应)
# 定义具体的挂载点规则
vim /etc/auto.shares

# 添加以下内容:
# * -rw,sync,fstype=nfs4 serverb:/shares/&
# ↑ ↑                      ↑
# 通配符 挂载选项          NFS服务器路径
#
# 参数详解:
# * = 匹配任意子目录名 (通配符)
# -rw,sync,fstype=nfs4 = 挂载选项 (详见下文)
# serverb = NFS服务器主机名
# :/shares/& = 导出路径 (&会被替换为实际访问的子目录名)

# 挂载选项说明:
# -rw = 读写挂载
# -sync = 同步写入 (数据更安全)
# -fstype=nfs4 = 指定文件系统类型为NFSv4

# 工作原理:
# 访问 /remote/operation → 自动挂载 serverb:/shares/operation

```

```

# 访问 /remote/production → 自动挂载 serverb:/shares/production
# 不访问时 → 自动卸载 (空闲超时后)

# 步骤2.4: 启动并启用autofs服务
# systemctl = systemd服务管理工具
# enable = 设置开机自启
# --now = 立即启动服务 (同时设置enable)
# autofs = 服务名称
systemctl enable --now autofs

# 等价于两条命令:
# systemctl start autofs      # 立即启动
# systemctl enable autofs     # 开机自启

# 步骤2.5: 测试autofs
# ls = 列出目录内容
# /remote/operation = 触发自动挂载
ls /remote/operation

# 首次访问时会自动挂载, 后续访问直接返回缓存内容

# 步骤2.6: 验证挂载
# df -h = 显示挂载信息
# grep remote = 过滤包含"remote"的行
df -h | grep remote

# 预期输出:
# serverb:/shares/operation 1.0G 100M 900M 10%
# /remote/operation
# ↑                ↑
# NFS资源          自动挂载到/remote下

# 步骤2.7: 查看autofs状态
# systemctl status = 显示服务运行状态
systemctl status autofs

# 输出说明:
# active (running) = 服务正在运行

```

相关练习: RH134V9

练习名称	对应章节	说明
练习: AUTOMOUNTING NETWORK-ATTACHED STORAGE	第09章 P290	自动挂载网络附加存储

总练习 (NFS网络存储)

练习名称	对应章节	说明
总练习：ACCESSING NETWORK-ATTACHED STORAGE	第09章 P295	访问网络附加存储综合练习

今日总结

重要命令清单

LVM:

SHELL

```
pvcreate          # 创建物理卷
vgcreate          # 创建卷组
lvcreate          # 创建逻辑卷
pvdisplay/vgdisplay/lvdisplay # 查看详细信息
pvs/vgs/lvs       # 简短查看
vgextend          # 扩展卷组
lvextend          # 扩展逻辑卷
xfs_growfs        # 扩展XFS文件系统
resize2fs         # 扩展EXT4文件系统
```

NFS:

SHELL

```
showmount -e server # 查看导出
mount server:/share /mnt # 挂载NFS
umount /mnt          # 卸载
```

autofs:

SHELL

```
systemctl enable --now autofs # 启用服务
vim /etc/auto.master.d/*.autofs # 主映射文件
vim /etc/auto.xxx              # 映射文件
```

常见任务

1. 创建LVM逻辑卷

SHELL

```
# 创建分区
parted /dev/vdb mkpart primary 1MiB 300MiB
parted /dev/vdb set 1 lvm on

# 创建LVM
pvcreate /dev/vdb1
vgcreate vname /dev/vdb1
lvcreate -L 200M -n lvname vname

# 格式化并挂载
mkfs.xfs /dev/vname/lvname
mkdir /mnt/data
mount /dev/vname/lvname /mnt/data
```

2. 扩展逻辑卷 (XFS)

SHELL

```
# 扩展LV
lvextend -L 400M /dev/vname/lvname

# 扩展文件系统
xfs_growfs /mnt/data
```

3. 扩展逻辑卷 (EXT4)

SHELL

```
# 扩展LV
lvextend -L 400M /dev/vname/lvname

# 扩展文件系统
resize2fs /dev/vname/lvname
```

4. 配置autofs间接映射

```
# 创建主映射
echo "/base /etc/auto.indirect" >
/etc/auto.master.d/indirect.autofs

# 创建映射文件
echo "share -rw server:/export/share" > /etc/auto.indirect

# 重启服务
systemctl restart autofs
```

关键命令速查

命令	作用	常用选项
<code>pvcreate</code>	创建物理卷	设备路径
<code>vgcreate</code>	创建卷组	-s PE大小
<code>lvcreate</code>	创建逻辑卷	-L 大小, -n 名称, -l LE数
<code>lvextend</code>	扩展逻辑卷	-L 大小, -l LE数
<code>xfs_growfs</code>	扩展XFS	挂载点
<code>resize2fs</code>	扩展EXT4	设备路径
<code>mount</code>	挂载NFS	host:/path
<code>showmount</code>	查看导出	-e
<code>systemctl</code>	管理autofs	enable, restart

感谢参与!

下节课见! 